

CONSTRAINTS in DBMS

What are Constraints?

- Constraints are rules applied on database tables.
- They ensure accuracy and consistency of data.
- Constraints prevent invalid data entry.

Definition

Constraint:

A rule used to limit the type of data that can be stored in a table.

Why Constraints are Important?

1. Maintain data accuracy
2. Prevent duplicate data
3. Avoid invalid values
4. Maintain relationships between tables
5. Improve database integrity

Types of Constraints in DBMS

1. NOT NULL
2. UNIQUE
3. PRIMARY KEY
4. FOREIGN KEY
5. CHECK
6. DEFAULT

Example Table

We use this Student table:

Student_ID Name Age Course Dept_ID

1. NOT NULL Constraint

Definition

- Ensures a column cannot contain NULL values.

Syntax

```
CREATE TABLE Student(  
    Student_ID INT,  
    Name VARCHAR(50) NOT NULL  
);
```

Explanation

- Name column must contain value.
- Empty value is not allowed.

Valid Query

```
INSERT INTO Student  
VALUES(101,'Ravi');
```

✓ Accepted

Invalid Query

```
INSERT INTO Student(Student_ID)  
VALUES(102);
```

✗ Error because Name is NULL.

Important Points

6. Prevents empty values
7. Used for mandatory fields

2. UNIQUE Constraint

Definition

- Ensures all values in a column are different.

Syntax

```
CREATE TABLE Student(  
    Email VARCHAR(100) UNIQUE  
);
```

Explanation

- Duplicate email IDs are not allowed.

Valid Query

```
INSERT INTO Student  
VALUES('ravi@gmail.com');
```

✓ Accepted

Invalid Query

```
INSERT INTO Student  
VALUES('ravi@gmail.com');
```

✗ Duplicate value error.

Important Points

8. Prevents duplicate values
9. Allows only unique entries

3. PRIMARY KEY Constraint

Definition

- Uniquely identifies each row in a table.

Syntax

```
CREATE TABLE Student(  
  Student_ID INT PRIMARY KEY,  
  Name VARCHAR(50)  
);
```

Explanation

- Student_ID must be:
 - Unique
 - NOT NULL

Valid Query

```
INSERT INTO Student  
VALUES(101,'Ravi');
```

✓ Accepted

Invalid Query

```
INSERT INTO Student  
VALUES(101,'Anu');
```

✗ Duplicate primary key error.

Important Points

10. Combines UNIQUE + NOT NULL
11. Only one primary key per table

4. FOREIGN KEY Constraint

Definition

- Creates relationship between two tables.

Example Tables

Department Table

```
CREATE TABLE Department(  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50)  
);
```

Student Table

```
CREATE TABLE Student(  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Dept_ID INT,  
    FOREIGN KEY(Dept_ID)  
    REFERENCES Department(Dept_ID)  
);
```

Explanation

- Dept_ID in Student table must exist in Department table.

Valid Query

```
INSERT INTO Department  
VALUES(1,'CSE');
```

```
INSERT INTO Student  
VALUES(101,'Ravi',1);
```

✓ Accepted

Invalid Query

```
INSERT INTO Student  
VALUES(102,'Anu',5);
```

✗ Dept_ID 5 does not exist.

Important Points

- 12.Maintains referential integrity
- 13.Connects tables

5. CHECK Constraint

Definition

- Ensures values satisfy a condition.

Syntax

```
CREATE TABLE Student(  
    Age INT CHECK(Age >=18)  
);
```

Explanation

- Age must be 18 or greater.

Valid Query

```
INSERT INTO Student  
VALUES(20);
```

✓ Accepted

Invalid Query

```
INSERT INTO Student  
VALUES(15);
```

✗ CHECK constraint violated.

Important Points

- 14. Restricts invalid values
- 15. Used for validation

6. DEFAULT Constraint

Definition

- Assigns default value automatically.

Syntax

```
CREATE TABLE Student(  
    Country VARCHAR(30)  
    DEFAULT 'India'  
);
```

Explanation

- If no value given, India is inserted automatically.

Query

```
INSERT INTO Student  
VALUES();
```

Output

Country

India

Important Points

- 16.Provides automatic values
- 17.Reduces manual entry

Combined Constraint Example

```
CREATE TABLE Student(  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Age INT CHECK(Age>=18),  
    Country VARCHAR(30) DEFAULT 'India'  
);
```

Explanation of Combined Table

Constraint	Purpose
PRIMARY KEY	Unique identification
NOT NULL	Mandatory field
UNIQUE	No duplicates
CHECK	Valid values
DEFAULT	Automatic value

Advantages of Constraints

- 18.Improves data integrity
- 19.Prevents invalid data
- 20.Maintains consistency
- 21.Improves database quality
- 22.Reduces redundancy
- 23.Maintains relationships
- 24.Improves security
- 25.Simplifies maintenance

Difference Between PRIMARY KEY and UNIQUE

PRIMARY KEY	UNIQUE
Cannot contain NULL	Can contain NULL
Only one per table	Multiple allowed
Uniquely identifies row	Ensures uniqueness

Additional Important Points

- 26.Constraints are applied during table creation.
- 27.Constraints maintain data correctness.
- 28.PRIMARY KEY uniquely identifies rows.
- 29.FOREIGN KEY connects tables.
- 30.CHECK validates conditions.
- 31.DEFAULT provides automatic value.
- 32.NOT NULL prevents empty values.
- 33.UNIQUE prevents duplicates.
- 34.Constraints improve reliability.
- 35.DBMS checks constraints automatically.

- 36. Violations produce errors.
- 37. Constraints are important in relational databases.
- 38. Foreign key references primary key.
- 39. Constraints improve consistency.
- 40. Constraints reduce data anomalies.

Real-Time Example

Banking System

Account Table

- Account_Number → PRIMARY KEY
- Balance → CHECK(Balance >= 0)

Customer Table

- Customer_ID → PRIMARY KEY

Transaction Table

- Account_Number → FOREIGN KEY

Short Definitions for Exams

Constraint

A rule applied on a table to maintain valid and consistent data.

Primary Key

An attribute that uniquely identifies each row.

Foreign Key

An attribute that references primary key in another table.

One-Mark Important Points

- 41. Constraints maintain data integrity.
- 42. PRIMARY KEY is unique and NOT NULL.

- 43.FOREIGN KEY maintains relationships.
- 44.CHECK validates conditions.
- 45.UNIQUE prevents duplicates.
- 46.DEFAULT provides automatic values.
- 47.NOT NULL prevents empty data.
- 48.Constraints improve consistency.
- 49.DBMS automatically checks constraints.
- 50.Constraints are essential in DBMS.

shyam-sudheer1602